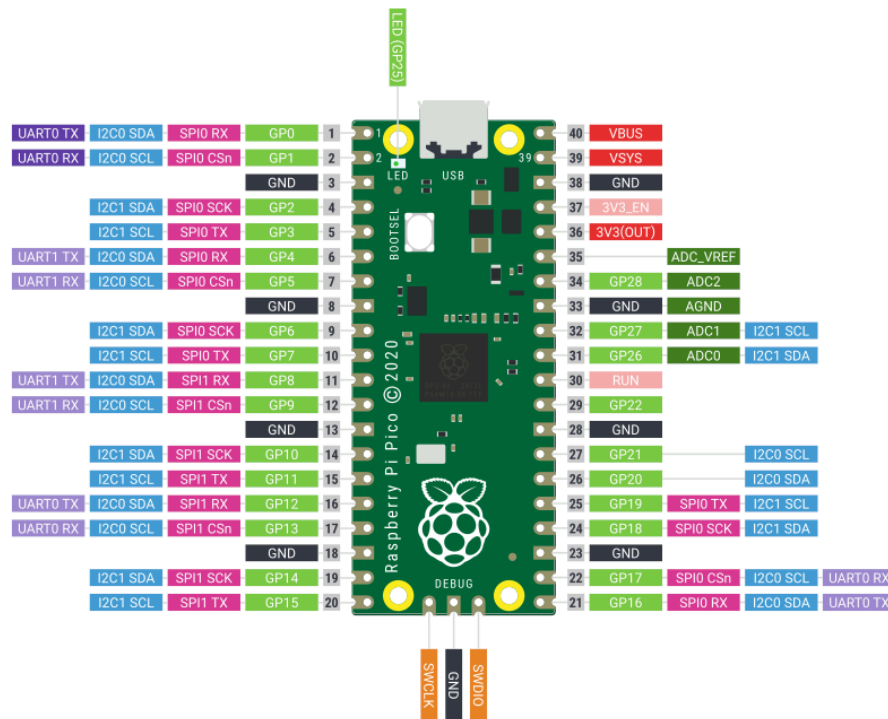


Ćwiczenie nr 1: Raspberry Pi Pico

MicroPython jest implementacją języka Python 3, która może być uruchomiona w systemach wbudowanych takich, jak Raspberry Pi Pico lub Raspberry Pi Pico W. Przed rozpoczęciem pracy z modułem Raspberry Pi Pico należy zapoznać się z instrukcją *Drag-and-Drop MicroPython*, która jest dostępna pod linkiem:

www.raspberrypi.com/documentation/microcontrollers/micropython.html



Zad 1. Napisać program, który co pół sekundy będzie zmieniał stan LED na płytce Raspberry Pi Pico. Dioda elektroluminescencyjna jest podłączona do GPIO 25.

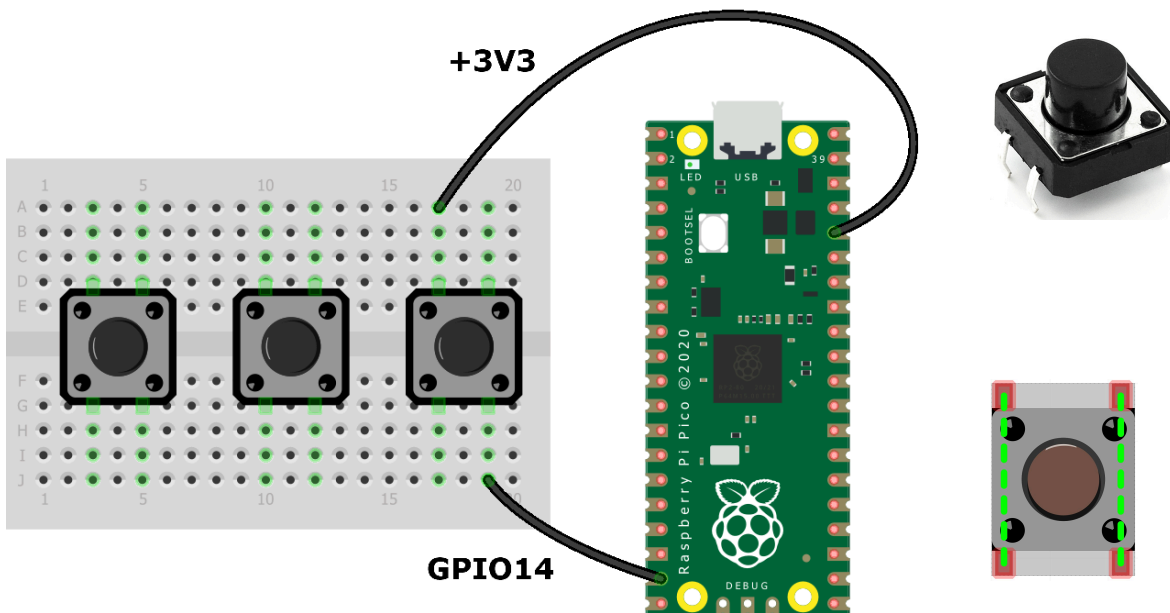
```
import machine
import utime as t

led_builtin = machine.Pin(25, machine.Pin.OUT)
i = 0
```

Ćwiczenie nr 1: Raspberry Pi Pico

```
while True:
    led_builtin.toggle()
    t.sleep(0.5)
    i+=1
    print(f'Current iteration is {i}')
```

Zad 2. Połączyć układ jak na rysunku. Przycisk jest podłączony do GPIO 14 (GP14). Napisać program, który wyśle komunikat *"Button pressed"* do komputera po naciśnięciu przycisku:



```
from machine import Pin
import utime

button = Pin(14, Pin.IN, Pin.PULL_DOWN)

while True:
    if button.value() == 1:
        print("Button pressed")
        utime.sleep(0.5)
```

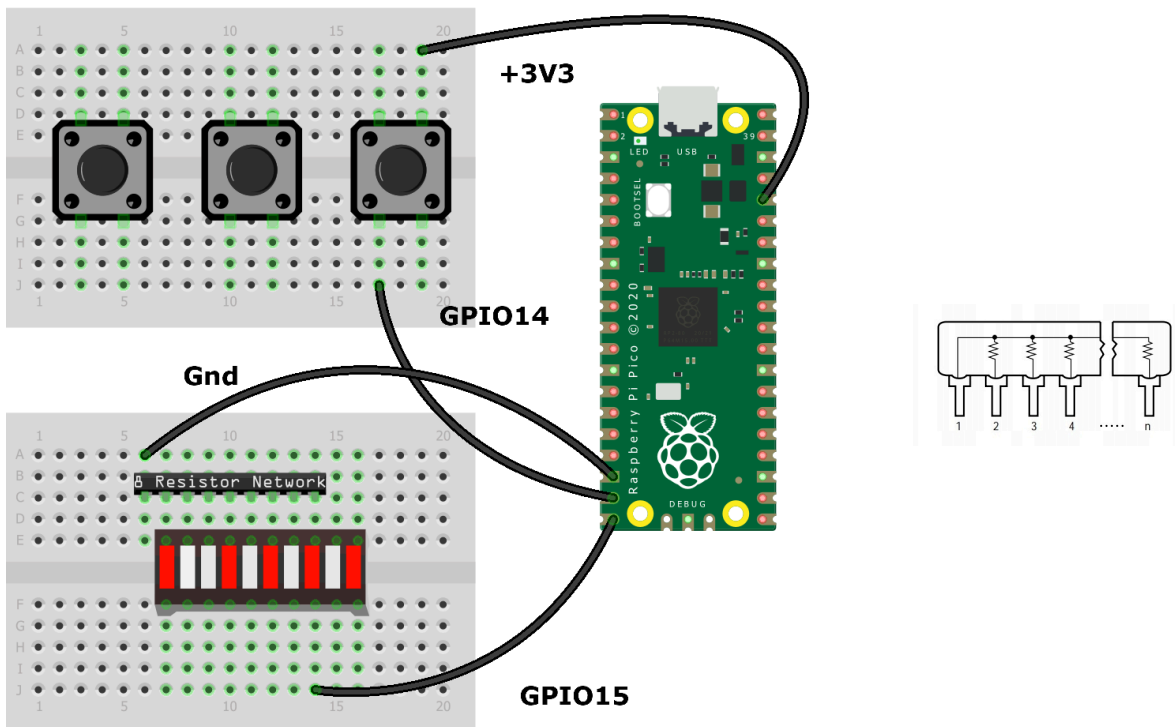
Pozwól użytkownikowi na podjęcie decyzji o treści komunikatu.

```
from machine import Pin
from utime import sleep

button = Pin(14, Pin.IN, Pin.PULL_DOWN)
message = input("Enter the message: ")
print('This is the message: ' + message)

while True:
    if button.value() == 1:
        print(message)
        sleep(0.5)
```

Zad 3. Połączyć układ jak na rysunku.



Jeżeli przycisk jest naciśnięty LED na płytce powinna być wyłączona. W przeciwnym wypadku LED powinna być włączona. Drabinka rezystorowa jest elementem, który zawiera w swojej strukturze 8 rezystorów, które posiadają 1 z wyprowadzeń połączone ze sobą.

```
from machine import Pin
import utime

button = Pin(14, Pin.IN, Pin.PULL_DOWN)
led = Pin(15, Pin.OUT)
led.value(1)

while True:
    if button.value() == 1:
        led.value(0)
    else:
        led.value(1)
        utime.sleep(0.1)
```

Zad 4. Korzystając z układu z zadania 3 napisać program, który pozwoli na wygenerowanie 4 impulsów światła po każdym naciśnięciu przycisku.

```
from machine import Pin
from utime import sleep

button = Pin(14, Pin.IN, Pin.PULL_DOWN)
led = Pin(15, Pin.OUT)

def led_blink(pin, times):
    for i in range(times):
        pin.toggle()
        sleep(0.5)
        pin.toggle()
        sleep(0.5)
led.value(0)
```

```
while True:
    if button.value() == 1:
        led_blink(led, 5)
        sleep(0.1)
```

Zad 5. Korzystając z układu, z zadania 3 napisać program implementujący funkcjonalność typowego włącznika oświetlania z przyciskiem monostabilnym.

```
from machine import Pin
from utime import sleep

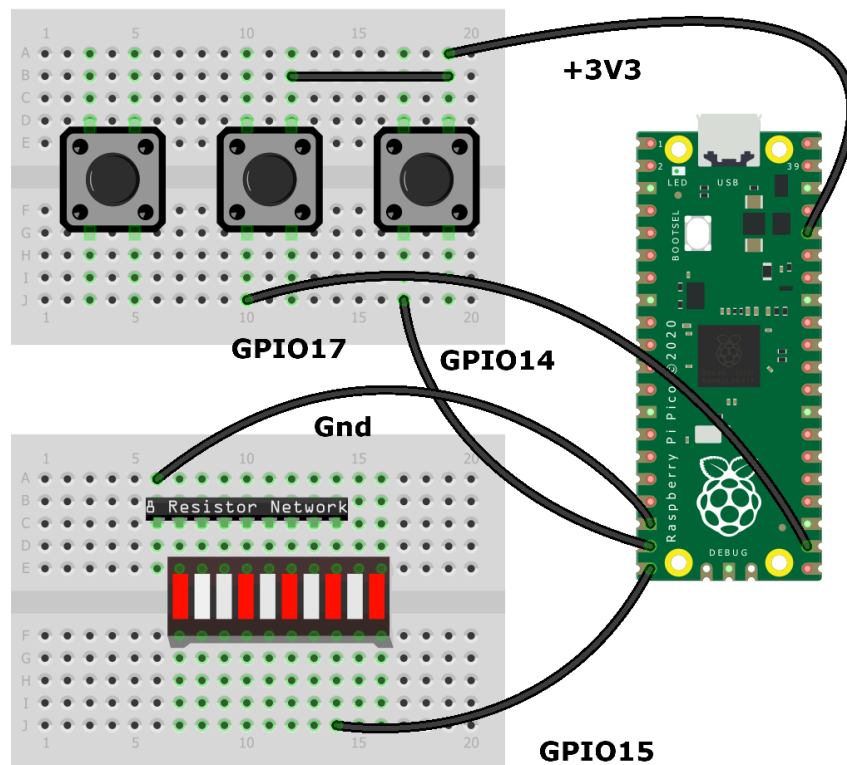
button = Pin(14, Pin.IN, Pin.PULL_DOWN)
light = Pin(15, Pin.OUT)

def button_clicked(pin):
    if pin.value() == 1:
        sleep(0.1)
        while pin.value() == 1:
            pass
        return True
    else:
        return False

light.value(0)

while True:
    if button_clicked(button):
        light.toggle()
```

Zad 6. Połączyć układ jak na rysunku. 1 z przycisków powinien załączać podłączoną do płytki Raspberry Pi Pico LED, a 2 wyłączać. Dioda na płycie powinna zmieniać swój stan 1 raz na sekundę.



```
from machine import Pin
from utime import sleep
import _thread

button_L = Pin(17, Pin.IN, Pin.PULL_DOWN)
button_R = Pin(14, Pin.IN, Pin.PULL_DOWN)
buildin_led = Pin(25, Pin.OUT)
external_led = Pin(15, Pin.OUT)

def led_blink():
    while True:
        buildin_led.toggle()
        sleep(0.5)

_thread.start_new_thread(led_blink, ())
```

```
def button_clicked(pin):
    if pin.value() == 1:
        sleep(0.1)
        while pin.value() == 1:
            pass
        return True
    else:
        return False

while True:
    if button_clicked(button_R):
        external_led.value(1)
        print("R click")
    if button_clicked(button_L):
        external_led.value(0)
        print("L click")
```

Save this project in the memory of a Raspberry Pi Pico board as `main.py`. Disconnect and then reconnect the USB cable. Observe results.

Zad 7. Komunikacja pomiędzy wątkami.

```
from utime import sleep
import _thread

def core0():
    global core1_takes_control
    counter = 0
    while True:
        for _ in range(3):
            print(f' Counter value = {counter}')
            counter += 1
            sleep(1)

        core1_takes_control = True

    print("Core 0 is waiting")
```

```
        while core1_takes_control:
            pass

def core1():
    global core1_takes_control
    counter = 100
    while True:
        print("Core 1 is waiting")
        while not core1_takes_control:
            pass

        for _ in range(3):
            print(f'Counter value = {counter}')
            counter += 10
            sleep(2)

        core1_takes_control = False

core1_takes_control = False

second_thread = _thread.start_new_thread(core1, ())
core0()
```

Zad 8. Współdzielenie zasobów.

Sprawdź działanie `lock.acquire()` oraz `lock.release()`.

```
from utime import sleep
import _thread

def core0():
    global lock
    while True:
        lock.acquire()
        print('C')
        sleep(0.5)
```



```
        print('O')
        sleep(0.5)
        print('R')
        sleep(0.5)
        print('E')
        sleep(0.5)
        print('O')
        sleep(0.5)
        lock.release()

def core1():
    global lock
    while True:
        lock.acquire()
        print('c')
        sleep(0.5)
        print('o')
        sleep(0.5)
        print('r')
        sleep(0.5)
        print('e')
        sleep(0.5)
        print('l')
        sleep(0.5)
        lock.release()

lock = _thread.allocate_lock()

_thread.start_new_thread(core1, ())
core0()
```

Zad 9. Sprawdzenie stanu blokady (polling the lock).

```
lock.acquire(True,1)
```

Parametry:

a. *blocking* (opcjonalne):

- `True` (default) - wątek czeka aż blokada będzie dostępna.

- `False` - wątek nie czeka; w przypadku braku dostępu wykonanie dalszej części programu.
- b. *timeout* (opcjonalne) - max czas (w sekundach) oczekiwania na zwolnienie blokady.
- `-1` (default) - czeka bez ograniczenia czasowego.

Zawracane wartości:

- `True` - blokada przechwycona.
- `False` - brak dostępu (`blocking=False` lub `timeout`).

```
from utime import sleep
import _thread

def core0():
    global lock
    while True:
        wait_counter = 0
        while not lock.acquire(False):
#           while not lock.acquire(True,1):
            wait_counter += 1
            print(f'Core0 counted to {wait_counter} while
waiting')

        print('C')
        sleep(0.5)
        print('O')
        sleep(0.5)
        print('R')
        sleep(0.5)
        print('E')
        sleep(0.5)
        print('O')
        sleep(0.5)
        lock.release()
```

```
def core1():
    global lock
    while True:
        lock.acquire()
        print('c')
        sleep(0.5)
        print('o')
        sleep(0.5)
        print('r')
        sleep(0.5)
        print('e')
        sleep(0.5)
        print('l')
        sleep(0.5)
        lock.release()

lock = _thread.allocate_lock()

_thread.start_new_thread(core1, ())
core0()
```

Zad 10. Podłącz wyświetlacz z rodziny *Pico-LCD-...* do płytki Raspberry Pi Pico.



Pobierz *Demo code* z sekcji *Resources* na odpowiedniej stronie, np:

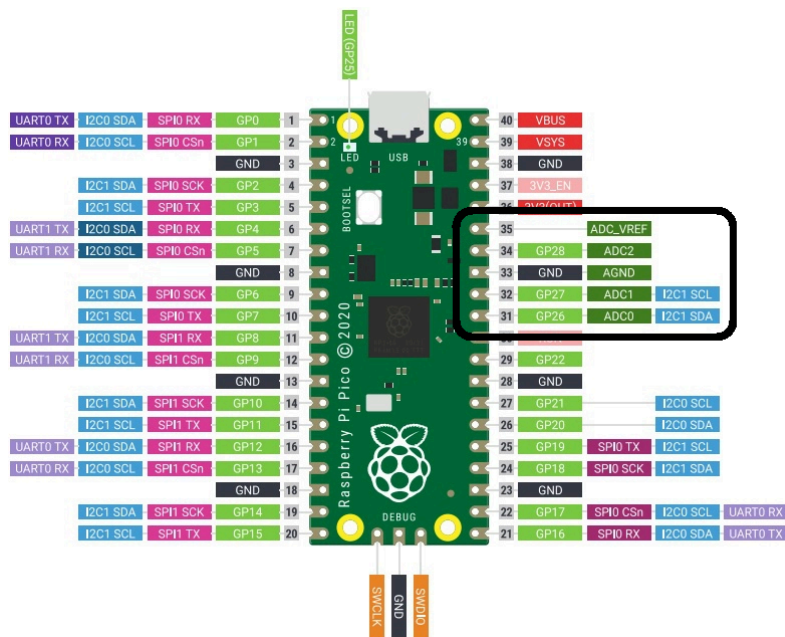
- www.waveshare.com/wiki/Pico-LCD-0.96

- www.waveshare.com/wiki/Pico-LCD-1.8

Wyświetl dowolny tekst na wyświetlaczu.

Zad 11. Przetwornik A/C.

Przetwornik A/C w RP2040 ma rozdzielczość 12 bitów, co oznacza, że kolejne próbki sygnału analogowego, podanego na wejście wybranego kanału, zostaną przekształcone w liczbę całkowitą z przedziału od 0 do 4095. Dzięki zastosowanej w module *machine* MicroPython'a technice nadpróbkowania rozdzielczość przetwarzania A/C została zwiększona do 16 bitów, co oznacza wartość liczbową od 0 do 65535. RP2040 posiada przetwornik 5-kanałowy. 4 kanały są dostępne poprzez wybrane linie GPIO: GP26, GP27, GP28 and GP29. Na płytkach Raspberry Pi Pico, pierwsze 3 z nich są dostępne dla użytkownika, a 4 umożliwia pomiar napięcia VSYS. Piąty kanał przetwornika A/C jest podłączony do czujnika temperatury wbudowanego w mikrokontroler RP2040.



Do odczytu wartości analogowej, proporcjonalnej do napięcia na danym kanale, można wykorzystać numer wyprowadzenia (pin) lub numer kanału:

```
adc = machine.ADC(26) # wyprowadzenie - GP26
adc = machine.ADC(0)  # kanał - odpowiada GP26
```

Skrypt w MicoPython'ie, który odczyty i wyśle do komputera wartość temperatury mierzonej przez wbudowany w RP2040 czujnik:

```
from machine import ADC
from utime import sleep

temp_sensor = ADC(4) # channel

while True:
    result = temp_sensor.read_u16()
    temperature = 27 - (result*3.3/65535 - 0.706)/0.001721
    print(temperature)
    sleep(2)
```

Zwykle, 0.706V oznacza 27 stopni Celsjusza, przy wartości współczynnika U/T równej -1.721mV (0.001721V) na stopień Celsjusza.

Zad 12. Wyświetl wartość temperatury wraz z jednostką na wyświetlaczu.

Do zaliczenia zadania wymagana jest prezentacja i przesłanie rozwiązania na adres wskazany w stopce instrukcji. Przed wysłaniem plik należy spakować lub zmienić rozszerzenie na txt.

Dla zainteresowanych:

1. Strona MicroPython:

micropython.org/download/rp2-pico/

2. Python wprowadzenie:

www.programiz.com/python-programming/first-program

3. Wprowadzenie do Raspberry Pi Pico:

projects.raspberrypi.org/en/projects/getting-started-with-the-pico/0